

计算机体系结构大作业

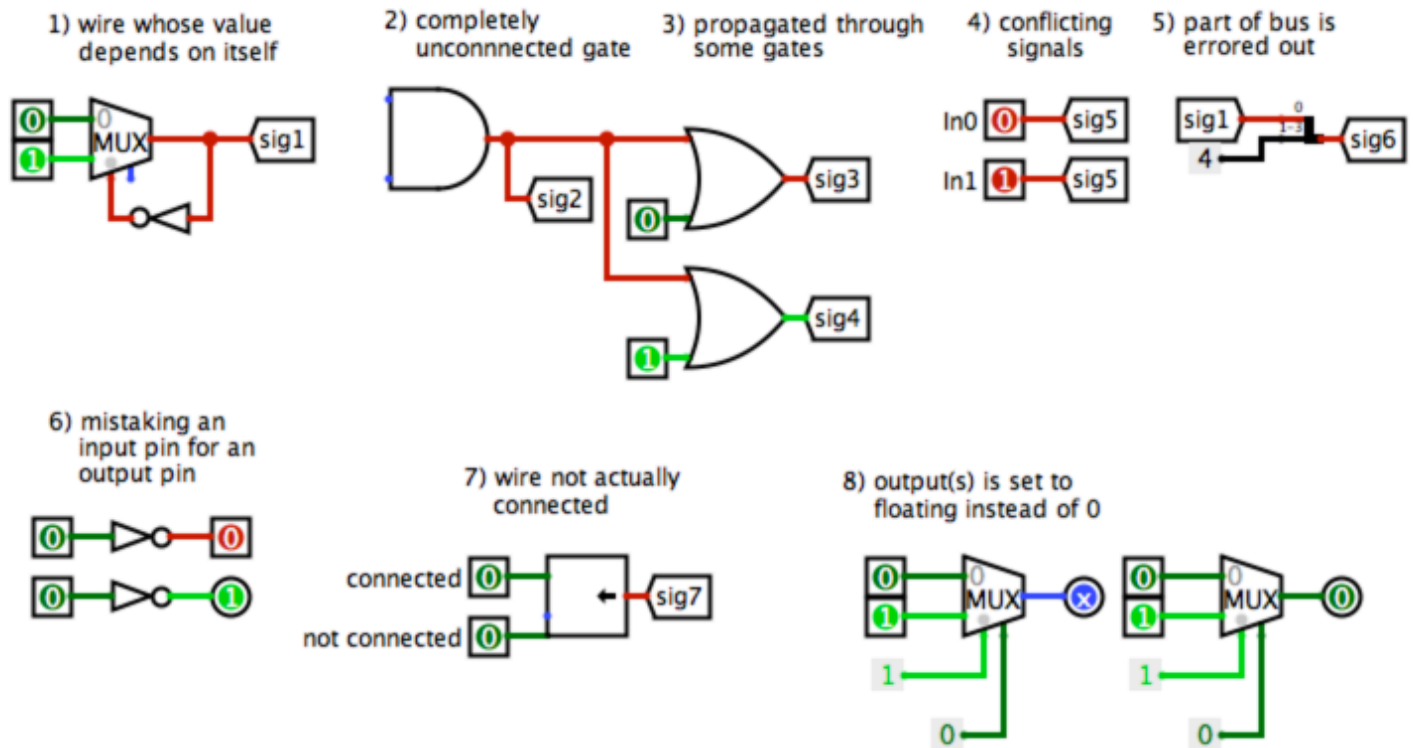
概述

本项目版权归属于CS61C。

在 Part A（任务 1-3）中，你将为一个基础的 RISC-V CPU 搭建 ALU 和寄存器文件，并实现执行 `addi` 指令的 CPU 数据通路。在 Part B（任务 4-5）中，你将使用这些组件（以及其它组件）来搭建一个能够运行实际 RISC-V 指令的工作 CPU。

提示与指南

- 请使用项目仓库中提供的 `logisim-evolution.jar`。
- 在本项目的所有部分，你**只允许**使用 Logisim 内置的以下库中的模块：Wiring（**除了** Transistor, Transmission Gate, POR, Pull Resistor, Power, Ground, POR, Do not connect）、Gates、Plexers、Arithmetic（**除了** Divider）、Memory（**除了** RAM, Random Generator）。注意允许使用 ROM。
- 大多数项目任务的健全性测试都包含在项目起始代码中。更多信息见每个任务的“测试”部分。
- **不要移动电路中的输入和输出引脚**，检查 `harness` 电路以及 `cpu.circ`，确保你的电路适合测试框架。
- 你可以创建新的子电路，但它们必须位于起始代码给定的文件中。你不能创建新的 `.circ` 文件。
- 你必须在所有 `.circ` 文件中为每个子电路使用**唯一的名称**。
- 你的提交必须**足够高效**：运行单个测试不应花费很长时间。
- Logisim 库参考可以通过在 Logisim 中进入 `Help->Library Reference` 找到。
- 一些常见的 Logisim 错误来源，供你调试时参考：



Part A：开始

对于本项目，你使用的电脑需要能够运行 Java 和 Python 3。本规范将使用 `python3` 作为示例命令；根据你的系统，你可能需要使用 `python` 或 `py` 代替。我们将使用 Logisim Evolution，一个基于 Java 的 GUI 程序，因此我们**强烈建议**在安装了 Java 并直接连接显示器的笔记本电脑或台式机上工作。通过 Hive SSH 会话进行窗口转发是不鼓励的，可能导致不稳定和普遍的头痛问题。

你可以从[起始代码仓库](#)中获取项目代码。

任务 1：算术逻辑单元 (ALU)

你的第一个任务是创建一个 ALU，支持我们的 ISA（下一节将详细描述）中所有指令所需的操作。请注意，我们像 RISC-V 处理无符号指令一样处理溢出，意味着我们忽略溢出。

我们在 `alu.circ` 中为你提供了一个 ALU 的框架。它有三个输入：

输入名称	位宽	描述
A	32	ALU 操作中用于输入 A 的数据
B	32	ALU 操作中用于输入 B 的数据
ALUSel	4	选择 ALU 应执行的操作（见下方操作列表及对应的开关值）

以及一个输出：

输出名称	位宽	描述
Result	32	ALU 操作的结果

以下是你需要实现的 ALU 操作列表，以及相应的 ALUSel 值。**所有操作都是必需的。**鼓励你使用 Logisim 内置模块来实现算术操作。请注意，由于 Logisim 中增加了有符号乘法器，使得 mulh 的实现与 mulhu 一样简单，因此 mulh 不再是额外加分项。

开关值	指令
0	add: Result = A + B
1	and: Result = A & B
2	or: Result = A B
3	xor: Result = A ^ B
4	srl: Result = (无符号) A >> B
5	sra: Result = (有符号) A >> B
6	sll: Result = A << B
7	slt: Result = (A < B (有符号)) ? 1 : 0
8	未使用
9	未使用
10	mul: Result = (有符号) (A * B)[31:0]
11	mulhu: Result = (A * B)[63:32]
12	sub: Result = A - B
13	bsel: Result = B
14	mulh: Result = (有符号) (A * B)[63:32]

在实现 mul 和 mulh 时，注意乘法模块有一个“进位输出”（加法器模块也有，但你不需要它）。

你可以对 alu.circ 进行任何修改，但输出必须遵守上述行为。如果你创建了额外的子电路，它们也必须在 alu.circ 中（你不能创建新的 .circ 文件）。此外，你的 ALU 必须能够放入提供的测试框架

`alu_harness.circ` 中。这意味着你应注意不要移动输入或输出。要验证你所做的更改没有破坏任何东西，可以打开 `alu_harness.circ` 并确保没有错误且电路功能正常。

提示：

- `add` 已经为你做好了；在实现其它模块时，可以随意使用类似的结构。
- 你可以将鼠标悬停在模块的输入/输出上，以获取有关该输入/输出的更详细信息。
- 如果你想了解每个模块的更多细节，请转到 `Help->Library Reference` 获取有关模块及其输入和输出的更多信息。
- 在实现 `sra` 和 `srl` 时，你可能会发现位拆分器或扩展器很有用。
- **使用隧道！** 它们将使你的布线更清晰、更容易跟踪，并减少遇到交叉线或意外错误的机会。
- 多路选择器 (MUX) 在决定要输出哪个模块的结果时可能很有用。换句话说，考虑简单地在所有模块中处理输入，然后选择你想要的输出。

ALU 测试

我们在 `tests/part_a/alu` 目录中提供了一组 ALU 健全性测试。运行该组的测试运行器将运行 ALU 测试，并在 `tests/part_a/alu/student_output` 目录中生成输出。

```
$ python3 test_runner.py part_a alu
```

我们还提供了 `binary_to_hex_alu.py`，它可以以可读格式解释此输出。要使用它，请执行以下操作：

```
$ cd tests/part_a/alu
$ python3 binary_to_hex_alu.py 输出文件路径
```

例如，要以可读格式查看 `reference_output/alu-add-ref.out`，你可以这样做：

```
$ cd tests/part_a/alu
$ python3 binary_to_hex_alu.py reference_output/alu-add-ref.out
```

如果你想查看你的输出与参考解决方案之间的差异，请将可读输出放入新的 `.out` 文件中并 `diff` 它们。例如，对于 `alu-add` 测试，你可以这样做：

```
$ cd tests/part_a/alu
$ python3 binary_to_hex_alu.py reference_output/alu-add-ref.out > reference.out
$ python3 binary_to_hex_alu.py student_output/alu-add-student.out > student.out
$ diff reference.out student.out
```

任务 2：寄存器文件 (RegFile)

RISC-V 架构有 32 个寄存器，在本项目中，我们将实现所有这些寄存器。为了便于调试，我们编写了寄存器文件，以暴露下面指定的 9 个端口。请确保这些寄存器的值连接到正确的输出。

你的 RegFile 应该能够写入或读取给定 RISC-V 指令中指定的寄存器，而不影响任何其他寄存器，除了 RegFile **不应** 写入 `x0`，零寄存器应始终具有值 `0x0`。你**不应**在 RegFile 中任何地方对时钟进行门控：时钟信号应始终**直接**连接到寄存器的时钟输入，而**不**经过任何组合逻辑。

暴露的寄存器及其对应编号如下：

寄存器编号	寄存器名称
x0	x0
x1	ra
x2	sp
x5	t0
x6	t1
x7	t2
x8	s0
x9	s1
x10	a0

我们在 `regfile.circ` 中为你提供了一个寄存器文件的框架。该寄存器文件电路有**六个**输入：

输入名称	位宽	描述
Clock	1	提供时钟的输入。此信号可以发送到子电路或直接连接到 Logisim 中存储单元的时钟输入，但不应用门控（即，不要反相，不要与任何东西“与”等）。
RegWEn	1	决定是否在下一个时钟上升沿将数据写入寄存器文件。
Read Register 1 (rs1)	5	决定哪个寄存器的值被发送到 Read Data 1 输出，见下文。

输入名称	位宽	描述
Read Register 2 (rs2)	5	决定哪个寄存器的值被发送到 Read Data 2 输出，见下文。
Write Register (rd)	5	决定在下一个时钟上升沿将 Write Data 的值设置给哪个寄存器，假设 RegWEn 为 1。
Write Data (wb)	32	决定在下一个时钟上升沿要写入由 Write Register 输入标识的寄存器的数据，假设 RegWEn 为 1。

寄存器文件还有以下输出：

输出名称	位宽	描述
rs1	32	由 Read Register 1 输入标识的寄存器的值。
rs2	32	由 Read Register 2 输入标识的寄存器的值。
ra Value	32	始终驱动为 ra 的值（这是一个 调试/测试 输出。）
sp Value	32	始终驱动为 sp 的值（这是一个 调试/测试 输出。）
t0 Value	32	始终驱动为 t0 的值（这是一个 调试/测试 输出。）
t1 Value	32	始终驱动为 t1 的值（这是一个 调试/测试 输出。）
t2 Value	32	始终驱动为 t2 的值（这是一个 调试/测试 输出。）
s0 Value	32	始终驱动为 s0 的值（这是一个 调试/测试 输出。）
s1 Value	32	始终驱动为 s1 的值（这是一个 调试/测试 输出。）
a0 Value	32	始终驱动为 a0 的值（这是一个 调试/测试 输出。）

regfile.circ 顶部的测试输出是为了测试和调试目的而存在的。如果你要实现一个真正的寄存器文件，你会省略这些输出。在我们的情况下，请确保它们正确包含——如果它们不正确，你将无法通过测试。

你可以对 regfile.circ 进行任何修改，但输出必须遵守上述行为。如果你创建了额外的子电路，它们也必须在 regfile.circ 中（你不能创建新的 .circ 文件）。此外，你提交的 regfile.circ 必须能够放入我们提供的 regfile_harness.circ 文件中。这意味着你应注意不要移动输入或输出。要验

证你所做的更改没有破坏任何东西，可以打开 `regfile_harness.circ` 并确保没有错误且电路功能正常。

RegFile 测试

我们在 `tests/part_a/regfile` 目录中提供了一组 RegFile 健全性测试。运行该组的测试运行器将运行 RegFile 测试，并在 `tests/part_a/regfile/student_output` 目录中生成输出。

```
$ python3 test_runner.py part_a regfile
```

我们还提供了 `binary_to_hex_regfile.py`，其工作方式与任务 1 中的 `binary_to_hex_alu.py` 类似。

任务 3： `addi` 指令

作为 Part A 的最后一个任务，你将实现一个能够执行一条指令：`addi` 的 CPU。

信息： 内存

内存单元（位于 `mem.circ` 中）已经为你完全实现，并连接到 `test_harness.circ` 中你 CPU 的输出上！`addi` 指令**不使用**内存单元，因此在 Part A 中你可以将这些输出保持未驱动状态。注意，提供的内存是按字寻址的，但具有**字节级**写使能。这意味着 `Write_En` 信号是 4 位宽，作为输入数据的写掩码。例如，如果 `Write_En` 是 `1'b1000`，则内存中被寻址字的最高有效字节将被覆盖。无论 `Write_En` 如何，`ReadData` 端口将始终返回指定地址的内存中的值。

以下是其输入和输出的简要总结：

信号名称	方向	位宽	描述
WriteAddr	输入	32	内存中要读/写的地址
WriteData	输入	32	要写入内存的值
Write_En	输入	4	对于写入内存的指令，这是写掩码；否则为 0
CLK	输入	1	由 CPU 的时钟输入驱动
ReadData	输出	32	存储在指定地址的数据的值

信息：分支比较器

框架中提供的分支比较器单元（位于 `branch_comp.circ`）尚未实现，但 `addi` 指令**不使用**分支比较器单元，因此在 Part A 中无需实现。

如果你感兴趣，以下是其输入和输出的简要总结：

信号名称	方向	位宽	描述
rs1	输入	32	要比较的第一个寄存器中的值
rs2	输入	32	要比较的第二个寄存器中的值
BrUn	输入	1	当需要无符号比较时为 1，需要有符号比较时为 0
BrEq	输出	1	如果两个值相等则为 1
BrLt	输出	1	如果 rs1 中的值小于 rs2 中的值则为 1

信息：立即数生成器

框架中提供的立即数生成器（“Imm Gen”）单元（位于 `imm_gen.circ`）尚未实现。`addi` 指令需要一个立即数生成器，但现在你可以将其硬连线以构造 `addi` 指令的立即数，而无需担心其他立即数类型。

要编辑此子电路，请编辑 `imm_gen.circ` 文件，而不是 `cpu.circ` 中的 `imm_gen`。注意，如果你修改此电路，则需要关闭并重新打开 `cpu.circ` 以在 CPU 中加载更改。

以下是其输入和输出的简要总结：

信号名称	方向	位宽	描述
inst	输入	32	正在执行的指令
ImmSel	输入	3	决定如何重构立即数的值
imm	输出	32	指令中立即数的值

信息：处理器

我们在 `cpu.circ` 中为你的处理器提供了一个框架。你将在构建数据通路时使用你自己实现的 ALU 和 RegFile。你负责从头开始构建整个数据通路。对于 Part A，你完成的处理器应支持 `addi` 指令，使用两级流水线，第一级为 IF，第二级为 ID、EX、MEM 和 WB。不过，开始时，我们建议你创建一个没有流水线的单级处理器。一旦工作正常，你可以修改处理器使其具有两级流水线。

你的处理器将位于包含内存单元的处理器框架 `test_harness.circ` 中。然后该处理器框架位于测试框架 `run.circ` 中，后者向处理器提供指令。你的处理器将输出指令的地址，并接受该地址的指令作为输入。它还将输出数据内存地址、数据内存写使能，并接受该地址的数据作为输入。本质上，这两个测试框架分别是你的数据内存和指令内存。我们建议你花些时间检查 `test_harness.circ` 和 `run.circ`，以确切了解发生了什么。`test_harness.circ` 将用于提供给您的健全性检查测试中，因此在测试和提交工作之前，请确保你的 CPU 适合该框架。你的处理器有来自框架的 3 个输入：

输入名称	位宽	描述
READ_DATA	32	由 WRITE_ADDRESS（见下文）标识的数据内存地址处的数据驱动。
INSTRUCTION	32	由 FETCH_ADDRESS（见下文）标识的指令内存地址处的指令驱动。
CLOCK	1	时钟输入。与寄存器文件一样，这可以发送到子电路（例如，你的寄存器文件的 CLK 输入）或直接连接到 Logisim 中存储单元的时钟输入，但不应用门控（即，不要反，不要与任何东西“与”等）。

你的处理器必须向第一级框架提供以下输出：

输出名称	位宽	描述
ra	32	由 ra 的内容驱动（用于测试）
sp	32	由 sp 的内容驱动（用于测试）
t0	32	由 t0 的内容驱动（用于测试）
t1	32	由 t1 的内容驱动（用于测试）
t2	32	由 t2 的内容驱动（用于测试）
s0	32	由 s0 的内容驱动（用于测试）
s1	32	由 s1 的内容驱动（用于测试）
a0	32	由 a0 的内容驱动（用于测试）

输出名称	位宽	描述
tohost	32	由 CSR 0x51E 的内容驱动（用于测试）
WRITE_ADDRESS	32	此输出用于选择在数据内存中读取/写入数据的地址。
WRITE_DATA	32	此输出用于向数据内存提供写入数据。
WRITE_ENABLE	4	此输出用于向数据内存提供写使能掩码。
fetch_addr	32	此输出用于选择在 INSTRUCTION 输入上呈现给处理器的指令。

与 ALU 和 RegFile 一样，注意**不要**移动输入或输出引脚！有关框架和测试的更多信息，请参见“测试”部分。

信息：控制逻辑

框架中提供的控制逻辑单元（ `control_logic.circ` ）尚未实现。设计你的控制逻辑单元可能是 Part B 中最大的挑战。对于 Part A，你可以为每个控制信号放置一个常数，因为 `addi` 是你将实现的唯一指令。在实现 `addi` 时，思考为了支持其他指令，你需要在哪些地方进行添加。

要编辑此子电路，请编辑 `control_logic.circ` 文件，而不是 `cpu.circ` 中的 `control_logic` 。注意，如果你修改此电路，则需要关闭并重新打开 `cpu.circ` 以在 CPU 中加载更改。

你可以根据控制逻辑的需要，向现有起始电路添加更多输入或输出。你也可以根据需要随意使用提供的端口中的任意数量。话虽如此，在此过程中请不要更改或删除任何现有端口。

单级 CPU：指南

回顾 CPU 流水线的五个阶段：

- 1. 指令获取 (IF)
- 2. 指令译码 (ID)
- 3. 执行 (EX)
- 4. 内存 (MEM)
- 5. 写回 (WB)

本指南将帮助你逐步完成 `addi` 指令的每个阶段。每个部分都会包含一些供你思考的问题，以及指向重要细节的提示，但不会确切告诉你如何实现该指令。

第 1 阶段：指令获取

在这个阶段，我们主要关心的是：如何获取当前指令？从课堂中我们知道，指令存储在指令存储器中，每条指令可以通过地址访问。

1. 项目中哪个文件包含你的指令存储器？它如何连接到你的 `cpu.circ` 文件？指令存储器是 `run.circ` 中的 ROM 模块。它向你的 CPU 提供一个名为 `INSTRUCTION` 的输入，并接收来自你 CPU 的一个输出。在你的 `cpu.circ` 文件中，该输出称为 `PROGRAM_COUNTER`。在 `run.circ` 中，它称为 `fetch_addr`。
2. 在你的 CPU 中，改变你作为 `PROGRAM_COUNTER` 输出的地址将如何影响指令输入？`run.circ` 输出到你的 CPU 的指令应该是指令存储器中地址为 `PROGRAM_COUNTER`（或 `fetch_addr`）处的指令。
3. 你如何知道 `PROGRAM_COUNTER` 应该是什么？`PROGRAM_COUNTER` 是当前正在执行的指令的地址，因此它保存在 PC 寄存器中。对于本项目，你的 PC 将从 0 开始，因为这是寄存器的默认值。
4. 对于没有任何跳转或分支的基本程序，PC 将如何逐行变化？PC 必须递增 1 条指令才能转到下一条指令，因为 PC 寄存器保存的地址代表要执行的指令。这意味着你的 PC 通常每行增加 4（假设没有分支或跳转）。

在 `cpu.circ` 中，我们提供了一个简单的 PC 寄存器实现——忽略跳转和分支。你将在项目的 Part B 中实现分支和跳转，但现在我们只关心能够运行 `addi` 指令。

请记住，我们最终将实现一个两级流水线处理器，因此 IF 阶段与其余阶段是分开的。什么电路将流水线的不同阶段分开？具体来说，是什么电路将 IF 与下一阶段分开？你需要添加任何东西吗？

第 2 阶段：指令译码

现在我们有来自 `instruction` 输入的指令，我们在指令译码步骤中根据你学过的 RISC-V 指令格式将其分解。

1. `addi` 是什么类型的指令？有哪些不同的位字段，每个字段需要哪些位？I 型。字段包括： - `imm [31-20]` - `rs1 [19-15]` - `funct3 [14-12]` - `rd [11-7]` - `opcode [6-0]`
2. 在 Logisim 中，你会使用 Splitter（拆分器）来**拆分**不同的位组。
3. 使用指令输入实现指令字段译码阶段。你应该使用隧道来标记和分组这些位。
4. 现在我们需要从相应的寄存器中获取数据，使用寄存器文件。哪些指令字段应该连接到寄存器文件？应该连接到寄存器文件的哪些输入？指令字段 `rs1` 需要连接到读寄存器 1。
5. 实现从寄存器文件读取。你需要从 Part A 引入你的 RegFile。记住连接时钟。
6. 立即数生成器需要做什么？对于 `addi`，立即数生成器从指令中接收 12 位，并产生一个有符号的 32 位立即数。你需要在立即数生成器子电路中实现此逻辑！。

第 3 阶段：执行

执行阶段是大多数指令进行计算的阶段。这也是我们将引入控制模块概念的地方。

1. 对于 `addi` 指令，你的 ALU 输入应该是什么？读数据 1 (`rs1`) 和立即数生成器产生的立即数。
2. 在 ALU 中，`ALU_Sel` 的作用是什么？它决定 ALU 将执行哪个操作。
3. 虽然现在可以将一个常数作为 `ALU_Sel`，但为什么随着你实现更多指令，这变得不可行？对于更多指令，ALU 的输入可能需要改变，因此你需要某种电路，根据正在执行的指令来改变 `ALU_Sel`。
4. 引入你的 ALU 并正确连接 ALU 输入。你需要连接时钟吗？为什么需要或不需要？

第 4 阶段：内存

内存阶段是使用存储指令写入内存和使用加载指令从内存读取的地方。由于 `addi` 指令不使用内存，我们在这里不会花太多时间。

此时，我们无法连接大多数输入，因为我们不知道它们应该来自哪里。

第 5 阶段：写回

写回阶段是操作结果保存回寄存器的地方。

1. `addi` 指令需要写回到寄存器吗？是的。`addi` 将 ALU 中加法计算的结果取出并写回到寄存器文件。
2. 让我们创建写回阶段，使其能够将 ALU 和 MEM 的输出都写入寄存器文件。稍后，当你实现分支/跳转时，可能需要向此 MUX 添加更多输入。但目前，我们需要在 ALU 和 MEM 输出之间选择，因为只有一根线最终能成为寄存器文件的输入。从 ALU 和 `READ_DATA` 各引出一根线，连接到 MUX。
3. 你应该使用什么作为 MUX 的选择输入？这个输入取决于什么？这个输入应该能够在三个 MUX 输入之间选择：(1) ALU，(2) `MEM [ READ_DATA ]`，以及 (3) `PC + 4`（你什么时候会用到这个？）决定写入回哪个输入的控制信号称为 `WB_Sel`。目前，`WB_Sel` 只能取一个值——即对于 `addi` 应有的值。
4. 现在我们已经理清了 MUX 的输入，我们需要连接输出。输出应该连接到什么地方？因为输出是你写入寄存器文件的数据，它应该连接到寄存器文件的 `Write Data` 输入。
5. 寄存器文件上还有两个输入对于写入数据很重要：`RegWEn` 和 `rd`。其中一个将来自指令译码阶段，另一个是你需要在 Part B 中设计的新的控制信号。请通过正确连接 `RegFile` 上的这些输入来完成写回阶段。

如果你正确地完成了所有步骤，你应该有一个适用于 `addi` 指令的单周期处理器。

运行 `python3 test_runner.py part_a addi_single` 看看它是否工作正常。

流水线化你的 CPU

现在是在你的 CPU 中实现流水线的时候了！对于本项目，你需要实现一个两级流水线。你将实现的两个阶段是：

1. 指令获取： 从指令存储器中取出一条指令。

2. 执行： 指令被译码、执行并提交（写回）。这是正常的五级 RISC-V 流水线中剩余四个阶段（ID、EX、MEM 和 WB）的组合。

由于所有控制和执行都在执行阶段处理，**你的处理器应该与单周期实现几乎没有区别，除了一个周期的启动延迟。** 然而，我们将强制要求两级流水线设计。需要考虑的一些问题：

- IF 和 EX 阶段会有相同还是不同的 PC 值？
- 你需要在流水线阶段之间存储 PC 吗？

你可能还会注意到一个引导问题：在第一个周期期间，位于流水线阶段之间的指令寄存器不会包含从内存加载的指令。我们如何处理这个问题？碰巧 Logisim 在复位时自动将寄存器设置为零；那么指令寄存器将包含一个 `nop`。我们允许你依赖 Logisim 的这种行为。记得进入 Simulate --> Reset Simulation (Cmd + R 或在 Windows 上按 Ctrl + R) 来复位你的处理器。

在对处理器进行流水线化之后，你应该能够通过 `python3 test_runner.py part_a addi_pipelined` 测试。注意，单级测试现在将失败，这是预期的行为（为什么？看看每个测试的参考输出，并思考流水线对不同阶段的影响）。

测试

理解测试

每个测试都是起始代码中包含的 `run.circ` 文件的一个副本，其 IMEM 中加载了指令。当你从命令行运行 `logisim-evolution` 时，时钟滴答，程序计数器递增，每个输出中的值被打印到标准输出。

以单级 `addi` 测试为例。它有 3 条 `addi` 指令

(`addi t0, x0, 5`、`addi t1, t0, 7`、`addi s0, t0, 9`)。在 Logisim Evolution 中打开 `tests/part_a/addi_pipelined/cpu-addi-pipelined.circ`，仔细观察测试文件的各个部分。在顶部，你会看到你的 CPU 连接到测试输出的地方。使用起始代码，你会看到所有 `xxxx`，如下所示；当你的 CPU 正常工作时，情况不应如此。你的 CPU 接收一个输入（指令），除了每个寄存器中的值外，它还有一个额外的输出：`fetch_addr`，即要从 IMEM 获取以便在下一个时钟周期执行的指令的地址。

注意不要移动 CPU 的任何输入/输出，也不要添加任何额外的输入/输出。这将改变 CPU 子电路的形状，因此测试文件中的连接可能无法再正常工作。

在 CPU 下方，你会看到指令存储器。3 条 `addi` 指令的十六进制数（0x00500293、0x00728313、0x00928413）已加载到指令存储器中。指令存储器接收一个输入（称为 `fetch_addr`）并输出该地址处的指令。`fetch_addr` 是一个 32 位值，但由于 Logisim Evolution 将 ROM 单元的大小限制为 $2^{16}B$ ，我们必须使用拆分器从 `fetch_addr` 中仅获取 14 位（忽略最低两位）。注意，`fetch_addr` 是一个**字节地址**，而不是字地址。

那么当时钟滴答时会发生什么？每次时钟滴答都会增加测试文件中名为 `Time_Step` 的输入。时钟将继续滴答，直到 `Time_Step` 等于该测试文件的停止常数（对于这个特定的测试文件，停止常数是 5）。此时，Logisim Evolution 命令行会将每个输出中的值打印到标准输出。我们的测试会将此输出与预期输出进行比较；如果你的输出不同，你将无法通过测试。

addi 测试

我们在起始代码中包含了 2 个用于 `addi` 指令的测试：一个用于单周期 CPU，一个用于流水线 CPU。你可以使用以下命令运行流水线测试（将 `pipelined` 替换为 `single` 以进行单周期测试）：

```
$ python3 test_runner.py part_a addi_pipelined # 用于流水线 CPU
```

你可以在 `tests/part_a/addi_pipelined/inputs` 中查看测试的 `.s`（RISC-V）和 `.hex`（机器码）文件。

我们包含了一个 Python 脚本来解释你的输出：`binary_to_hex_cpu.py`，其工作方式与前面任务中的 `binary_to_hex_alu.py` 和 `binary_to_hex_regfile.py` 类似。要使用该脚本查看你的 CPU 输出，请运行：

```
$ cd tests/part_a/addi_pipelined
$ python3 binary_to_hex_cpu.py student_output/CPU-addi-pipelined-student.out
```

或者，要查看参考输出，请运行：

```
$ cd tests/part_a/addi_pipelined
$ python3 binary_to_hex_cpu.py reference_output/CPU-addi-pipelined-ref.out
```

Part B

任务 4：更多指令

在任务 3 中，你搭建了一个能够执行 `addi` 指令的基本流水线 CPU。现在，你将实现更多指令的支持！

指令集架构 (ISA)

我们将仅根据下面列出的指令对你的 CPU 实现进行评分。你的 CPU 必须支持这些指令，但只要你不对所需指令的实现产生影响，随意实现任何额外指令都可以。实现额外指令不会影响你的分数。

指令	类型	Opcode	Funct3	Funct7/Immediate	操作
<code>add rd, rs1, rs2</code>	R	0x33	0x0	0x00	$R[rd] \leftarrow R[rs1] + R[rs2]$
<code>mul rd, rs1, rs2</code>			0x0	0x01	$R[rd] \leftarrow (R[rs1] * R[rs2])[31:0]$
<code>sub rd, rs1, rs2</code>			0x0	0x20	$R[rd] \leftarrow R[rs1] - R[rs2]$
<code>sll rd, rs1, rs2</code>			0x1	0x00	$R[rd] \leftarrow R[rs1] \ll R[rs2]$
<code>mulh rd, rs1, rs2</code>			0x1	0x01	$R[rd] \leftarrow (R[rs1] * R[rs2])[63:32]$
<code>mulhu rd, rs1, rs2</code>			0x3	0x01	(无符号) $R[rd] \leftarrow (R[rs1] * R[rs2])[63:32]$
<code>slt rd, rs1, rs2</code>			0x2	0x00	$R[rd] \leftarrow (R[rs1] < R[rs2]) ? 1 : 0$ (有符号)
<code>xor rd, rs1, rs2</code>			0x4	0x00	$R[rd] \leftarrow R[rs1] \wedge R[rs2]$
<code>srl rd, rs1, rs2</code>			0x5	0x00	(无符号) $R[rd] \leftarrow R[rs1] \gg R[rs2]$

指令	类型	Opcode	Funct3	Funct7/Immediate	操作
sra rd, rs1, rs2			0x5	0x20	(有符号) $R[rd] \leftarrow R[rs1] \gg R[rs2]$
or rd, rs1, rs2			0x6	0x00	$R[rd] \leftarrow R[rs1]$
and rd, rs1, rs2			0x7	0x00	$R[rd] \leftarrow R[rs1] \& R[rs2]$
lb rd, offset(rs1)	I	0x03	0x0		$R[rd] \leftarrow$ $\text{SignExt}(\text{Mem}(R[rs1] +$ $\text{offset, 字节}))$
lh rd, offset(rs1)			0x1		$R[rd] \leftarrow$ $\text{SignExt}(\text{Mem}(R[rs1] +$ $\text{offset, 半字}))$
lw rd, offset(rs1)			0x2		$R[rd] \leftarrow \text{Mem}(R[rs1] +$ $\text{offset, 字})$
addi rd, rs1, imm	I	0x13	0x0		$R[rd] \leftarrow R[rs1] + \text{imm}$
slli rd, rs1, imm			0x1	0x00	$R[rd] \leftarrow R[rs1] \ll \text{imm}$
slti rd, rs1, imm			0x2		$R[rd] \leftarrow (R[rs1] < \text{imm}) ? 1$: 0
xori rd, rs1, imm			0x4		$R[rd] \leftarrow R[rs1] \wedge \text{imm}$
srli rd, rs1, imm			0x5	0x00	$R[rd] \leftarrow R[rs1] \gg \text{imm}$ (逻辑)
srai rd, rs1, imm			0x5	0x20	$R[rd] \leftarrow R[rs1] \gg \text{imm}$ (算术)
ori rd, rs1, imm			0x6		$R[rd] \leftarrow R[rs1]$

指令	类型	Opcode	Funct3	Funct7/Immediate	操作
andi rd, rs1, imm			0x7		$R[rd] \leftarrow R[rs1] \& \text{imm}$
sb rs2, offset(rs1)	S	0x23	0x0		$\text{Mem}(R[rs1] + \text{offset}) \leftarrow R[rs2][7:0]$
sh rs2, offset(rs1)			0x1		$\text{Mem}(R[rs1] + \text{offset}) \leftarrow R[rs2][15:0]$
sw rs2, offset(rs1)			0x2		$\text{Mem}(R[rs1] + \text{offset}) \leftarrow R[rs2]$
beq rs1, rs2, offset	SB	0x63	0x0		if($R[rs1] == R[rs2]$) $PC \leftarrow PC + \{\text{offset}, 1b0\}$
bne rs1, rs2, offset			0x1		if($R[rs1] != R[rs2]$) $PC \leftarrow PC + \{\text{offset}, 1b0\}$
blt rs1, rs2, offset			0x4		if($R[rs1] < R[rs2]$) (有符号) $PC \leftarrow PC + \{\text{offset}, 1b0\}$
bge rs1, rs2, offset			0x5		if($R[rs1] \geq R[rs2]$) (有符号) $PC \leftarrow PC + \{\text{offset}, 1b0\}$
bltu rs1, rs2, offset			0x6		if($R[rs1] < R[rs2]$) (无符号) $PC \leftarrow PC + \{\text{offset}, 1b0\}$
bgeu rs1, rs2, offset			0x7		if($R[rs1] \geq R[rs2]$) (无符号) $PC \leftarrow PC + \{\text{offset}, 1b0\}$
auipc rd, offset	U	0x17			$R[rd] \leftarrow PC + \{\text{offset}, 12b0\}$
lui rd, offset	U	0x37			$R[rd] \leftarrow \{\text{offset}, 12b0\}$
jal rd, imm	UJ	0x6f			$R[rd] \leftarrow PC + 4, PC \leftarrow PC + \{\text{imm}, 1b0\}$
jalr rd, rs1, imm	I	0x67	0x0		$R[rd] \leftarrow PC + 4, PC \leftarrow R[rs1] + \{\text{imm}\}$
csw rd, csr, rs1	I	0x73	0x1		$\text{CSR}[\text{csr}] \leftarrow R[rs1]$

指令	类型	Opcode	Funct3	Funct7/Immediate	操作
cswi rd, csr, uimm	I	0x73	0x5		CSR[csr] ← {uimm}

信息：内存

内存单元（位于 `mem.circ` 中）已经为你完全实现，并连接到 `test_harness.circ` 中你 CPU 的输出上！你**不得**将 `mem.circ` 添加到你的 CPU 中，这样做会导致自动评分器失败，并且你将得不到分数。

注意，你提供给内存的地址是一个字节地址，但内存返回的是一个完整的字。内存单元忽略你提供给它的地址的最低两位，并将其输入视为**字地址**而非**字节地址**。例如，如果你输入 32 位地址 `0x0000_1007`，它将被视为字地址 `0x0000_1004`，并且你将得到地址 `0x0000_1004`、`0x0000_1005`、`0x0000_1006` 和 `0x0000_1007` 处的 4 个字节作为输出。

注意，对于 `lw`、`lh`、`sh`、`sw` 指令，RISC-V ISA 支持非对齐访问，但实现起来很复杂。在本项目中，我们只实现**对齐**的内存访问。这意味着操作仅在不超过内存中连续字的边界时才有定义。此类操作的一个例子是任何 `lw` 或 `sw`，它们在 4 的倍数的地址上进行操作。由于地址是 4 的倍数，并且我们加载 4 个字节的一个字，获取的总内存不会超过内存中连续字的边界。你**不得**实现非对齐访问；你可能需要使用停顿，这将导致你的输出与我们的预期输出不匹配（对你的分数不利）。

请记住，内存也是**字节级**写使能的。这意味着 `Write_En` 信号是 4 位宽，作为输入数据的写掩码。例如，如果 `Write_En` 是 `1'b1000`，则内存中被寻址字的最高有效字节将被覆盖。无论 `Write_En` 如何，`ReadData` 端口将始终返回指定地址的内存中的值。

再次提醒，以下是其输入和输出的简要总结：

信号名称	方向	位宽	描述
WriteAddr	输入	32	内存中要读/写的地址
WriteData	输入	32	要写入内存的值
Write_En	输入	4	对于写入内存的指令，这是写掩码；否则为 0
CLK	输入	1	由 CPU 的时钟输入驱动
ReadData	输出	32	存储在指定地址的数据的值

信息：分支比较器

分支比较器单元（位于 `branch_comp.circ`）比较两个值并输出将用于分支决策的控制信号。你需要为此电路实现逻辑。

要编辑此子电路，请编辑 `branch_comp.circ` 文件，而不是 `cpu.circ` 中的 `branch_comp`。注意，如果你修改此电路，则需要关闭并重新打开 `cpu.circ` 以在 CPU 中加载更改。

再次提醒，以下是其输入和输出的简要总结：

信号名称	方向	位宽	描述
rs1	输入	32	要比较的第一个寄存器中的值
rs2	输入	32	要比较的第二个寄存器中的值
BrUn	输入	1	当需要无符号比较时为 1，需要有符号比较时为 0
BrEq	输出	1	如果两个值相等则为 1
BrLt	输出	1	如果 rs1 中的值小于 rs2 中的值则为 1

信息：立即数生成器

立即数生成器（“Imm Gen”）单元（位于 `imm_gen.circ`）从 I、S、B、U 和 J 类型指令中提取适当的立即数。记住，在 RISC-V 中，所有离开立即数生成器的立即数都是 32 位且经过符号扩展的！请参阅下表了解每个立即数应如何格式化：

31	30	20	19	12	11	10	5	4	1	0	
— inst[31] —						inst[30:25]	inst[24:21]	inst[20]	I-immediate		
— inst[31] —						inst[30:25]	inst[11:8]	inst[7]	S-immediate		
— inst[31] —					inst[7]	inst[30:25]	inst[11:8]	0	SB-immediate		
inst[31]	inst[30:20]		inst[19:12]		— 0 —						U-immediate
— inst[31] —			inst[19:12]		inst[20]	inst[30:25]	inst[24:21]	0	UJ-immediate		

要编辑此子电路，请编辑 `imm_gen.circ` 文件，而不是 `cpu.circ` 中的 `imm_gen`。注意，如果你修改此电路，则需要关闭并重新打开 `cpu.circ` 以在 CPU 中加载更改。

再次提醒，以下是其输入和输出的简要总结：

信号名称	方向	位宽	描述
inst	输入	32	正在执行的指令
ImmSel	输入	3	决定如何重构立即数的值
imm	输出	32	指令中立即数的值

信息：控制逻辑

框架中提供的控制逻辑单元（ `control_logic.circ` ）基于课堂和讨论中使用的五级 CPU 的控制逻辑单元。为了正确识别每条指令，控制信号在本项目中起着非常重要的作用。尝试用不同类型的指令遍历数据通路；当你看到 MUX 或其他组件时，思考你需要为该指令设置什么选择器/使能值。

欢迎你根据控制逻辑的需要，向现有起始电路添加更多输入或输出。你也可以根据需要随意使用提供的端口中的任意数量。话虽如此，在此过程中请不要更改或删除任何现有端口。

实现控制逻辑以便从指令中提取 `opcode` / `funct3` / `funct7` 并相应地设置控制信号，主要有两种方法。

推荐的方法是**硬连线控制**，这通常是像 MIPS 和 RISC-V 这样的 RISC 架构的首选方法。硬连线控制使用“与”、“或”和“非”门（以及我们从这些门中学到的各种组件，如 MUX 和 DEMUX）来产生适当的控制信号。一个**指令译码器**接收一条指令，并输出该指令的所有控制信号。

另一种方法是使用 **ROM 控制**。处理器实现的每条指令都映射到只读存储器 (ROM) 单元中的一个地址。在该地址的 ROM 中，存放着该指令的控制字。一个**地址译码器**接收一条指令，并输出该指令控制字的地址。这种方法常见于像 Intel x86-64 这样的 CISC 架构中，并且在现实生活中提供了一定的灵活性，因为它可以通过更改 ROM 的内容来重新编程。

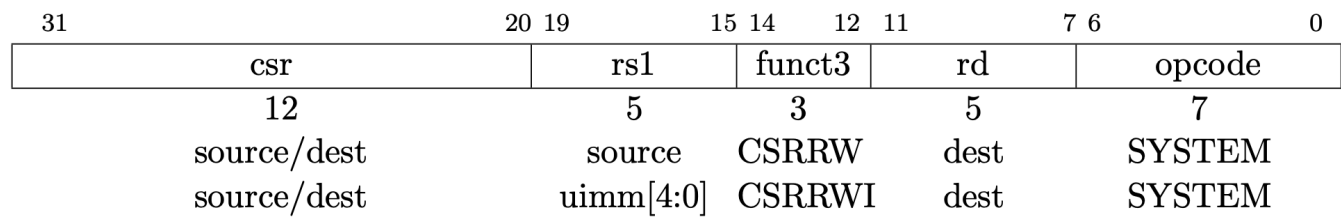
要编辑此子电路，请编辑 `control_logic.circ` 文件，而不是 `cpu.circ` 中的 `control_logic` 。注意，如果你修改此电路，则需要关闭并重新打开 `cpu.circ` 以在 CPU 中加载更改。

信息：控制状态寄存器 (CSR)

控制状态寄存器 (CSR) 保存有关机器指令结果的额外信息，并且通常独立于寄存器文件和内存存储。在你的处理器中，你将向其中一个 CSR 写入输出，该 CSR 将由更复杂的测试平台监控。

以下是**2 条 CSR 指令**，你需要实现它们。注意，虽然有 212 个可能的 CSR 地址，但我们只会使用其中一个（tohost = 0x51E）。注意，立即数形式使用编码在 rs1 字段中的 5 位零扩展立即数 (uimm)。

- 1. csrwr tohost, t2 （csrrw x0,csr,rs1 的简写，其中 csr=0x51E）
- 2. csrwi tohost, 1 （csrrwi x0,csr,uimm 的简写，其中 csr=0x51E）



控制状态寄存器单元框架（csr.circ）与起始代码一起提供。在此单元中，你需要实现 tohost CSR 寄存器以及写入逻辑。**请不要更改给定的连接，因为它会干扰测试平台的结果！**

如果你想了解更多关于 CSR 的信息，可以参考 [RISC-V 规范](#) 的第 9 章。

以下是其输入和输出的简要总结：

信号名称	方向	位宽	描述
CSR_address	输入	12	输入的 CSR 寄存器地址
CSR_din	输入	32	要写入指定 CSR 寄存器的值
CSR_WE	输入	1	写使能
clk	输入	1	时钟输入
tohost	输出	32	tohost 寄存器的输出

信息：处理器

主 CPU 电路（位于 cpu.circ 中）实现了主要数据通路，并将所有子电路（ALU、分支比较器、控制逻辑、立即数生成器、内存和 RegFile）连接在一起。在 Part A 中，你在 CPU 中实现了一个简单的两级流水线。由于 Part B 需要支持分支和跳转指令，你将需要处理分支时发生的控制冒险。

如果分支发生跳转，紧跟在分支或跳转之后的指令不会被执行。这使得你的任务稍微复杂一些。当你在执行阶段确定有一条分支或跳转指令时，你已经访问了指令存储器并取出了（可能）错误的指令。因此，如果正在执行的指令是跳转或条件满足的分支，你需要“杀死”正在被取指的指令。

对于本项目，指令的杀死**必须**通过将一个 `nop` 多路复用到指令流中，并将该 `nop` 送入执行阶段来实现，而不是使用取出的指令。注意，`0x00000013`，即 `addi x0, x0, 0` 是一条 `nop` 指令；其他 `nop` 指令也可以。如果分支发生跳转（条件满足），你应该杀死指令（否则不杀死）。对每种类型的跳转都要进行杀死操作。

不要通过在 IF 阶段计算分支偏移量来解决此问题。

由于所有控制和执行都在执行阶段处理，**你的处理器应该与单周期实现几乎没有区别，除了一个周期的启动延迟以及分支/跳转的延迟。**然而，我们将强制要求两级流水线设计。需要考虑的一些问题：

- IF 和 EX 阶段会有相同还是不同的 PC 值？
- 你需要在流水线阶段之间存储 PC 吗？
- 要将 `nop` 多路复用到指令流中，你是将其放在指令寄存器**之前**还是**之后**？
- 当 EX 阶段执行 `nop` 时，接下来应该请求什么地址？这和正常情况不同吗？

在第一个周期期间，位于流水线阶段之间的指令寄存器不会包含从内存加载的指令。我们如何处理这个问题？碰Logisim 在复位时自动将寄存器设置为零；那么指令寄存器将包含一个 `nop`。我们允许你依赖 Logisim 的这种行为。记得进入 Simulate --> Reset Simulation (Ctrl+R) 来复位你的处理器。

CPU 测试：流水线健全性测试

我们在 `tests/part_b/pipelined` 中为你的流水线 CPU 提供了一组基本的健全性测试。

```
$ python3 test_runner.py part_b pipelined
```

你可以在 `tests/part_b/pipelined/inputs` 中查看测试的 `.s`（RISC-V）和 `.hex`（机器码）文件。

我们还提供了 `binary_to_hex_cpu.py`，其工作方式与任务 3 中的 `binary_to_hex_cpu.py` 类似。

任务 5：自定义测试

自动评分器测试主要分为 3 类：**单元测试**、**集成测试**和**边缘情况测试**。对于 Part B，我们不会透露所有的自动评分器测试，但你应该能够自己创建非常接近的近似值来测试你的 CPU。

单元测试：单元测试用单条指令测试你的数据通路，以确保每条单独的指令都已实现并按预期工作。你应该为需要实现的每一条指令编写一个不同的单元测试，并确保彻底测试该指令的各种可能性。例如，`slt` 的单元测试应包含 `rs1 < rs2`、`rs1 > rs2` 以及 `rs1 == rs2` 的情况。

集成测试：通过单元测试后，转向组合使用多个功能的测试。尝试各种简单的 RISC-V 程序，这些程序运行单个函数；如果工作正常，你的 CPU 应该能够处理它们。随意尝试使用 `riscv-gcc` 将 C 程序编译为 RISC-V，但要注意我们使用的指令集有限（例如，你没有 `ecall` 指令）。我们建议你根据在实验、讨论、项目和考试中看到的内容，自己尝试编写简单的函数。

边缘情况测试：边缘情况测试尝试你通常不会期望的输入，这可能会在某些情况下触发错误。你应该寻找哪些边缘情况？一个小提示：我们的两大类边缘情况来自内存操作和分支/跳转操作（一些测试名称是“mem-full”和“br-jump-edge”）。思考这些操作可能出错的所有不同方式。

创建自定义测试

我们包含了一个脚本（`create-test.py`），它使用 Venus 来帮助你从 RISC-V 汇编生成测试电路，生成自定义测试的过程如下：

1. 为测试想出 RISC-V 汇编指令，并将它们保存在 `tests/part_b/custom/inputs` 文件夹中一个以 `.s` 结尾的文件中。该文件的名称将成为你的测试名称。注意，你的测试**必须**以 `.s` 结尾，并且它**必须**位于 `tests/part_b/custom/inputs` 文件夹中。如果你有更多测试，请重复此步骤。
 - 例如 `tests/part_b/custom/inputs/sll-slli.s`、`tests/part_b/custom/inputs/slt.s`
2. 如果你有更多测试，请重复步骤 1。
3. 要为你的测试生成测试电路，请运行 `create-test.py` 脚本：

```
$ cd tests/part_b/custom
$ python3 create-test.py inputs/sll-slli.s inputs/slt.s
```

这应该会生成几个新文件：

```
tests/part_b/custom:
- <测试名称>.circ # 你的测试的新电路
- inputs/<测试名称>.s # 你编写的测试文件
- inputs/<测试名称>.hex # 你编写的测试文件的机器码
- reference_outputs/<测试名称>-ref.out # 你的测试的参考输出
```

4. 现在你可以运行你的测试了！

```
$ python3 test_runner.py part_b custom
```

如果你想检查运行测试的电路，可以通过打开 CPU-<测试名称>.circ 文件，右键单击 CPU，选择“view main”，然后使用 Cmd + T（Windows 上为 Ctrl + T）手动滴答时钟每个周期来模拟它。如果你不记得如何模拟电路，请参考 Logisim 实验。我们强烈建议你在模拟测试时探查你的电路，以观察你的电路对各种输入的反应（也许这可以为你编写新测试提供思路）。

如果你只想模拟特定周期数的代码，可以使用 -n 标志：

```
$ python3 create_test.py -n <周期数> <测试名称>.s
```

如果你想解码你的输出，请使用提供的 binary_to_hex_cpu.py 。注意，因为你实现的是两级流水线处理器，并且第一条指令在第二个时钟周期的上升沿写入，所以你的指令的效果会有 2 条指令的延迟。例如，假设你编写了一个只有一条指令的测试： `addi t0, x0, 1` 。如果你检查参考输出，你会看到 `t0` 直到第三个周期才改变。